

SENTINEL

— LLM —

*Security Evaluation and Threat Intelligence
for Neural Language Models*

F-PIA · PIAP-LLM

Prompt Injection Analysis and Prevention
for Large Language Models

Diego Urbano, Juan Velasco

Framework Metodológico de Ciberseguridad
para Sistemas Basados en IA (LLM)

Ciberseguridad: White Hat & Blue Team Strategies

2026

Índice

1	Introducción al Framework	2
1.1	Definición	2
1.2	Propósito	2
2	Estructura del Framework (F-PIA)	3
2.1	Fase 1: Prevención	3
2.2	Fase 2: Identificación	3
2.3	Fase 3: Análisis	3
2.4	Fase 4: Mitigación	3
2.5	Matriz Base de Evaluación	3
3	Objetivo del Framework	4
4	Alcance	4
5	Metodología de Pruebas	5
5.1	Enfoque Metodológico	5
5.2	Matriz de Evaluación	6
6	Categorías de Prueba	6
6.1	Categoría de Reconocimiento	7
6.2	Categoría de Prompt Injection	7
7	Buenas Prácticas	7
7.1	Controles Técnicos	7
7.2	Controles Operativos	8
7.3	Controles Organizacionales	8
8	Casos de Prueba: Repositorio de Guías y Scripts del PIAP-LLM	8
8.1	Herramientas de Pentesting y Pruebas de Prompt Injection	9
8.2	Laboratorios Guiados – GitHub	9
9	Recursos Útiles	9
10	Recomendaciones Finales	10
11	Glosario de Términos	10
	Bibliografía	10

1. Introducción al Framework

El **PIAP-LLM** (*Prompt Injection Analysis and Prevention for Large Language Models*) es un framework metodológico diseñado para abordar de manera integral los riesgos asociados a ataques de *prompt injection* en sistemas basados en modelos de lenguaje (LLM).

Este marco propone un enfoque estructurado que permite:

- Prevenir vulnerabilidades en el diseño de prompts.
- Identificar comportamientos anómalos o manipulaciones del modelo.
- Analizar el impacto de entradas maliciosas.
- Definir estrategias de mitigación y control.

Su aplicación abarca entornos como chatbots, sistemas RAG (*Retrieval-Augmented Generation*), agentes inteligentes e integraciones con APIs.

RAG (Retrieval-Augmented Generation) — Generación Aumentada por Recuperación:

- **Retrieval:** Recuperación (búsqueda de información externa).
- **Augmented:** Aumentada (enriquecida con datos adicionales).
- **Generation:** Generación (respuesta del modelo).

1.1. Definición

Definición

El **PIAP-LLM** es un marco metodológico estructurado que permite identificar, evaluar, documentar y mitigar vulnerabilidades asociadas a ataques de *Prompt Injection* en aplicaciones que utilizan modelos de lenguaje (LLM), garantizando principios de confidencialidad, integridad y control del comportamiento del sistema.

1.2. Propósito

Proveer a las organizaciones una guía técnica para:

- Detectar vulnerabilidades en prompts y flujos conversacionales.
- Evaluar el impacto de ataques sobre sistemas LLM.
- Estandarizar pruebas de seguridad en IA.
- Definir controles y contramedidas.
- Fortalecer el SSDLC (*Secure Software Development Life Cycle*) en sistemas con IA.

2. Estructura del Framework (F-PIA)

2.1. Fase 1: Prevención

- Diseño seguro de prompts (*prompt hardening*).
- Separación de contexto (*system* vs *user* vs *tool*).
- Uso de validadores y sanitización.
- Implementación de políticas de control (*guardrails*).

2.2. Fase 2: Identificación

1. Detección de entradas maliciosas.
2. Clasificación de tipos de *prompt injection*:
 - Directa.
 - Indirecta (RAG).
 - *Role override*.
 - *Data exfiltration*.
 - *Instruction override*.

2.3. Fase 3: Análisis

- Evaluación del comportamiento del modelo.
- Comparación: respuesta esperada vs. respuesta obtenida.
- Medición de impacto: Bajo / Medio / Alto / Crítico.
- Identificación de fallos en controles.

2.4. Fase 4: Mitigación

- Implementación de filtros.
- Refuerzo de instrucciones del sistema.
- Uso de capas intermedias (*middleware* de seguridad).
- Monitoreo continuo (logs, alertas, SIEM).

2.5. Matriz Base de Evaluación

- Flujos automatizados impulsados por IA.

El framework cubre tanto entornos:

- Web.
- Backend (APIs REST).
- Microservicios.
- Arquitecturas multicapa.

Incluyendo escenarios donde el modelo interactúa con bases de datos, archivos, herramientas externas y sistemas empresariales.

5. Metodología de Pruebas

Casos controlados, entradas maliciosas simuladas, evaluación de respuestas y clasificación del riesgo.

El F-PIA define una metodología de evaluación basada en **testing adversarial controlado**, evitando la explotación real de vulnerabilidades y enfocándose en la simulación de escenarios de ataque.

5.1. Enfoque Metodológico

1. Reconocimiento (*Reconnaissance*)

Fase inicial del pentesting ético donde se recopila información del objetivo mediante herramientas especializadas y técnicas de reconocimiento pasivo y activo en ciberseguridad, orientadas a la recolección de información relevante del sistema objetivo en un entorno controlado y autorizado, garantizando la no afectación de la disponibilidad e integridad del servicio, y alineado con las buenas prácticas definidas por OWASP.

2. Definición de casos de prueba

- Basados en patrones de *prompt injection*.
- Uso de lenguaje controlado (no destructivo).

3. Ejecución de pruebas

- En entornos controlados (laboratorios, *staging*).
- Simulación de entradas maliciosas.

4. Evaluación de respuestas

- Comparación: respuesta esperada vs. obtenida.

- Identificación de comportamientos anómalos.

5. Clasificación del riesgo

- Bajo / Medio / Alto / Crítico.
- Basado en impacto y explotación.

6. Registro de resultados

- Documentación estructurada.
- Evidencias de pruebas.

Esta metodología se alinea con el enfoque de **testing basado en patrones lingüísticos**, donde el ataque no es código, sino instrucciones interpretadas por el modelo.

5.2. Matriz de Evaluación

Tipo de ataque, descripción, entrada usada, respuesta esperada, respuesta obtenida, impacto, severidad y recomendación.

El PIAP-LLM establece una matriz estructurada para documentar y analizar cada prueba de seguridad:

Descripción
Identificador único del caso de prueba
Ej: Categoría de <i>prompt injection</i>
Explicación del escenario
Ej: Prompt de prueba
Comportamiento correcto del sistema
Resultado real
Consecuencia del comportamiento
Nivel de riesgo
Acción de mitigación

6. Categorías de Prueba

6.1. Categoría de Reconocimiento

Reconocimiento pasivo y activo para la recolección de información relevante del sistema objetivo bajo pentesting ético con herramientas especializadas y técnicas de reconocimiento en ciberseguridad.

6.2. Categoría de Prompt Injection

Prompt injection directa, indirecta, fuga de información, manipulación de rol, evasión de políticas, contaminación de contexto y pruebas en RAG.

El PIAP-LLM clasifica los escenarios de prueba según tipos de ataque identificados:

- **Prompt Injection directa:**
Entrada maliciosa introducida directamente por el usuario.
- **Prompt Injection indirecta (RAG):**
Inyección a través de documentos, archivos o fuentes externas.
- **Fuga de información (*Data Exfiltration*):**
Obtención de datos internos o sensibles.
- **Manipulación de rol (*Role Override* / Escalada):**
Alteración de privilegios del modelo.
- **Evasión de políticas (*Instruction Override* / Jailbreak):**
Intento de ignorar restricciones del sistema.
- **Contaminación de contexto (*Context Poisoning*):**
Alteración del contexto para inducir respuestas erróneas.
- **Pruebas en entornos RAG:**
Evaluación de inyección en *pipelines* de recuperación de información.

Estas categorías están alineadas con la taxonomía de ataques y el **OWASP Top 10** para LLM y aplicaciones de IA generativa.

7. Buenas Prácticas

Validación de entradas, separación de instrucciones, control de contexto, filtros, monitoreo, trazabilidad, pentesting ético y revisión humana.

El PIAP-LLM establece un conjunto de buenas prácticas para fortalecer la seguridad en sistemas con LLM:

7.1. Controles Técnicos

- Pentesting ético con herramientas especializadas.

- Validación y sanitización de entradas.
- Separación estricta de:
 - `system prompt`.
 - `user input`.
 - Contexto externo.
- Implementación de filtros de entrada y salida.
- Control de formato de respuestas.
- Aplicación del principio de mínimo privilegio.

7.2. Controles Operativos

- Monitoreo continuo (logs, alertas, SIEM).
- Trazabilidad de interacciones.
- Auditoría periódica de prompts.

7.3. Controles Organizacionales

- Revisión humana (*Human-in-the-loop*).
- Pruebas adversariales periódicas.
- Capacitación en seguridad de IA.

8. Casos de Prueba: Repositorio de Guías y Scripts del PIAP-LLM

El framework PIAP-LLM incorpora un repositorio de artefactos técnicos que permite la ejecución reproducible de pruebas de seguridad. Estos recursos incluyen scripts, guías en formato PDF, configuraciones de laboratorio y colecciones de pruebas, almacenados en un repositorio versionado (GitHub).

Los artefactos permiten:

- Ejecutar pruebas controladas con herramientas de pentesting ético (ofensivo–defensivo).
- Replicar escenarios de *Prompt Injection*.
- Validar resultados del framework con matrices de evaluación.
- Facilitar auditorías y procesos de formación.

8.1. Herramientas de Pentesting y Pruebas de Prompt Injection

Los casos de prueba del framework PIAP-LLM se enfocan en el análisis de vulnerabilidades de: <https://www.latinoamericacomparte.com/>

Las herramientas seleccionadas son:

- ZAP (Zed Attack Proxy).
- *[Por definir]*.
- Prompt Injection Scripts Python.
- Prompt Injection Patterns.

Las matrices de evaluación contienen:

Descripción
Identificador único del caso de prueba
Categoría de <i>prompt injection</i>
Explicación del escenario
Prompt malicioso utilizado en la prueba
Comportamiento correcto esperado del sistema
Resultado real observado
Consecuencia del comportamiento
Nivel de riesgo
Acción de mitigación

8.2. Laboratorios Guiados – GitHub

Repositorio: *[Por definir]*

- **Pentesting:** *[Por definir]*
- **Prompt Injection** (Scripts Python + Prompt Injection Patterns): *[Por definir]*

9. Recursos Útiles

- Documentación de las herramientas.

- OWASP — <https://owasp.org>

10. Recomendaciones Finales

A considerar — subjetivas.

11. Glosario de Términos

A considerar.

LLM *Large Language Model.* Modelo de lenguaje de gran escala entrenado con grandes volúmenes de texto.

Prompt Injection

Técnica de ataque que introduce instrucciones maliciosas en el contexto de un modelo de lenguaje para alterar su comportamiento.

RAG *Retrieval-Augmented Generation.* Arquitectura que combina recuperación de información externa con generación de respuestas por parte del modelo.

SSDLC *Secure Software Development Life Cycle.* Ciclo de vida de desarrollo de software con enfoque en seguridad.

SIEM *Security Information and Event Management.* Sistema de gestión de información y eventos de seguridad.

OWASP *Open Web Application Security Project.* Organización sin ánimo de lucro que promueve la seguridad en aplicaciones web.

Jailbreak

Técnica para evadir las restricciones o políticas de un modelo de lenguaje.

Guardrails

Controles o restricciones implementados en sistemas LLM para limitar comportamientos no deseados.

Pentesting

Prueba de penetración. Evaluación de seguridad mediante simulación de ataques controlados.

Bibliografía

Pendiente de completar.

Referencias sugeridas:

- OWASP LLM Top 10 — <https://owasp.org/www-project-top-10-for-large-language-models/>
- NIST AI Risk Management Framework — <https://www.nist.gov/system/files/documents/2023/01/26/AI%20RMF%201.0.pdf>
- Perez, F. & Ribeiro, I. (2022). *Ignore Previous Prompt: Attack Techniques for Language Models*.